Step 3 — Run Them in Parallel and Compile (final) — Solution

> Back to the step · Hint 1 · Hint 2

Story: `RunnableParallel` to run all three chains at once, `run_meeting` to compile the results, `main.py`, and the integration test. **If not:** running three independent model calls one after another is three times the wait for no reason.

A sample transcript (`samples/standup.txt`)

```
# samples/standup.txt
Sara: Let's ship v2 on Friday. Everyone good with that? ... Agreed.
Ali: We should retire the legacy API next sprint.
Sara: Yes, decided. Ali, can you book the launch venue?
Ali: Sure. Sara, you'll send the Q3 budget to finance?
Sara: Done by tomorrow. We still need someone to write the migration guide.
```

The parallel pipeline

```
# src/pipeline.py
# Run the three extractor chains at the same time with RunnableParallel (they
# don't depend on each other), then fold the results into one MeetingReport.
from langchain_core.runnables import RunnableParallel

from src.agents.actions import build_actions_chain
from src.agents.decisions import build_decisions_chain
from src.agents.summary import build_summary_chain
from src.report import compile_report
from src.utils.logger import get_logger

def build_pipeline(model):
    return RunnableParallel(
        summary=build_summary_chain(model),
        decisions=build_decisions_chain(model),
        actions=build_actions_chain(model),
    )
```

```
def run_meeting(model, transcript):
    logger = get_logger(__name__)
    pipeline = build_pipeline(model)
    parts = pipeline.invoke({"transcript": transcript})
    report = compile_report(parts)
    logger.info(
        f"meeting processed: {len(report.decisions)} decision(s), "
        f"{len(report.action_items)} action(s)"
    )
    return report
```

`RunnableParallel` gives each chain the same input, runs them concurrently, and returns a dict keyed by the names you gave — `{"summary": ..., "decisions": ..., "actions": ...}` — which is exactly the shape `compile_report` (Step 1) expects.

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_meeting
from src.report import format_report


def main():
    if len(sys.argv) < 2:
        print("usage: python main.py <transcript-file>")
        return
    path = sys.argv[1]
    with open(path) as transcript_file:
        transcript = transcript_file.read()

    config = load_config()
    model = get_model(config)
    report = run_meeting(model, transcript)
    print()
    print(format_report(report))


if __name__ == "__main__":
    main()
```

The integration test (real model)

```
# tests/integration/test_live_meeting.py
# Real model. Run from the repo root:
#   python -m tests.integration.test_live_meeting
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_meeting
from src.report import format_report

TRANSCRIPT = (
    "Sara: Let's ship v2 on Friday. Agreed.\n"
    "Sara: Ali, can you book the launch venue?\n"
    "Ali: Sure."
)


def test_report_names_decision_and_action():
    config = load_config()
    model = get_model(config)
    report = run_meeting(model, TRANSCRIPT)
    text = format_report(report).lower()
    ok = "v2" in text and "venue" in text
    print(f"case 1 (names decision + action) -> {'OK' if ok else 'WRONG'}")


def test_has_at_least_one_of_each():
    config = load_config()
    model = get_model(config)
    report = run_meeting(model, TRANSCRIPT)
    ok = len(report.decisions) >= 1 and len(report.action_items) >= 1
    print(f"case 2 (>= 1 decision + action)  -> {'OK' if ok else 'WRONG'}")


if __name__ == "__main__":
    test_report_names_decision_and_action()
    test_has_at_least_one_of_each()
```

Expected run:

```
$ python main.py samples/standup.txt

Summary:
  The team agreed to ship v2 on Friday and retire the legacy API.
```

```
Decisions:
  - Ship v2 on Friday
  - Retire the legacy API next sprint

Action items:
  - Send the Q3 budget to finance (Sara)
  - Book the launch venue (Ali)
  - Write the migration guide (unassigned)
```

That's the whole project: three narrow specialists, run at the same time because they're independent, compiled by deterministic code into one report. It's the supervisor-of-specialists idea in its simplest, most useful shape — and the `RunnableParallel` + compile pattern scales straight up to the bigger multi-agent systems in Projects 4 and 11.